

拼图游戏

软件设计说明书

版本：V1.0

日期：2026年6月

本文档包含拼图游戏的软件架构设计、
模块详细设计及核心算法说明

目录

一、引言

- 1.1 编写目的
- 1.2 项目概述
- 1.3 术语定义

二、系统总体设计

- 2.1 架构概述
- 2.2 技术栈
- 2.3 目录结构

三、核心功能模块设计

- 3.1 主控模块 Main.cs
- 3.2 事件系统
- 3.3 数据管理器 datamgr.cs
- 3.4 玩家数据 PlayerData.cs
- 3.5 拼图管理器 picmgr.cs
- 3.6 拖拽系统 DraggableGridItem.cs
- 3.7 卡片系统 card.cs
- 3.8 边框系统 BorderSpriteLoader.cs

四、用户界面设计

- 4.1 章节选择界面
- 4.2 游戏界面
- 4.3 设置界面

五、数据配置设计

- 5.1 章节配置
- 5.2 关卡配置

六、核心算法说明

- 6.1 拼图打乱算法
- 6.2 连通组检测算法

6.3 位置交换算法

6.4 胜利检测算法

6.5 边框刷新算法

七、运行环境

一、引言

1.1 编写目的

本文档详细描述了拼图游戏软件的总体架构设计、模块划分、核心算法及实现方案，为软件著作权申请提供技术文档支撑。本文档适用于软件开发人员、测试人员及相关评审人员。

1.2 项目概述

本软件是一款基于 Unity 引擎开发的拼图游戏，运行于 iOS 和 Android 移动平台。玩家通过拖拽拼图碎片完成图片复原，游戏提供多种难度等级和丰富的关卡设计。核心玩法包括：图片网格切割、碎片随机打乱、拖拽交互、连通组整体移动、自动边框刷新、胜利检测等。

1.3 术语定义

拼图碎片：构成完整图片的最小单元，每个碎片对应原图的一个矩形区域。

连通组：在拼图网格中位置相邻且逻辑关系正确的碎片集合，可整体拖拽移动。

体力：玩家进行游戏所需消耗的资源，随时间自动恢复。

关卡：拼图的基本单位，包含待拼合的图片及切割参数。

章节：关卡的集合，同一章节内的关卡共享一张预览图片。

Luban：配置表工具，将 Excel 配置导出为 JSON 供游戏读取。

二、系统总体设计

2.1 架构概述

本游戏采用 Unity 引擎组件化架构，分为三大层级：表现层（UI 模块）、逻辑层（核心玩法）、数据层（配置与持久化）。各模块通过自定义事件系统实现松耦合通信，降低模块间依赖。

表现层负责所有用户界面交互，包括章节选择、游戏主界面、设置界面等，各界面继承自统一的 `frmbase` 基类。逻辑层实现拼图核心玩法，包括图片切割、碎片打乱、拖拽交互、连通组检测、位置交换、边框刷新、胜利判定等。数据层基于 `Luban` 框架加载 JSON 配置，通过 `LitJson` 实现玩家数据的本地持久化存储。

2.2 技术栈

游戏引擎：Unity 2022.3 LTS
脚本语言：C# (.NET Standard 2.1)
动画引擎：DOTween
JSON 序列化：LitJson
配置表：Luban
UI 系统：UGUI + TextMeshPro
目标平台：iOS / Android

2.3 目录结构

Assets/	
Scenes/SampleScene.unity	# 主场景
Scripts/	
Main.cs	# 入口 + 事件系统
card.cs	# 关卡卡片
picmgr.cs	# 拼图管理器
datamgr.cs	# 数据管理器
PlayerData.cs	# 玩家数据
DraggableGridItem.cs	# 拖拽系统
BorderSpriteLoader.cs	# 边框加载
DebugManager.cs	# 调试管理
ui/	
frmbase.cs	# UI 基类
frm_chapter.cs	# 章节界面
frm_game.cs	# 游戏界面
frm_setup.cs	# 设置界面
Resources/Card/	# 边框贴图
Plugins/	# 第三方库

三、核心功能模块设计

3.1 主控模块 Main.cs

Main 是游戏启动入口，采用单例模式全局访问。负责：初始化 DOTween 引擎；提供事件注册/派发/注销机制（RegistEvent / DispEvent / UnRegistEvent）；管理游戏暂停/恢复状态；提供关卡重新开始功能。

```
public class Main : MonoBehaviour {

    public static Main inst;

    delegate object registfun(object p);

    static Dictionary<string, List<registfun>> evs = new();

    public static void RegistEvent(string ev, registfun f) {

        if (!evs.ContainsKey(ev)) evs[ev] = new();

        evs[ev].Add(f);

    }

    public static object DispEvent(string ev, object p = null) {

        if (!evs.ContainsKey(ev)) return null;

        foreach (var fn in evs[ev]) { var r = fn(p); if (r != null) return r; }

        return null;

    }

    public static void UnRegistEvent(string ev, registfun f) {

        if (evs.ContainsKey(ev)) evs[ev].Remove(f);

    }

    void Awake() { inst = this; DOTween.Init();

        RegistEvent("game_restart", p => { RestartLevel(); return null; });

    void Start() {

        Screen.SetResolution((int)(1920*Screen.width/Screen.height), 1920, false);

        DispEvent("gamebegin");

    }

    public static void PauseGame() { Time.timeScale = 0; }

    public static void ResumeGame() { Time.timeScale = 1; }

    public static void RestartLevel() {

        if (!PlayerData.gd.hasEnoughpower(10)) return;

        PlayerData.gd.消耗power(10);

    }

}
```

3.2 事件系统

事件系统采用观察者模式，维护以事件名为键、处理函数列表为值的字典。事件被派发时依次调用所有注册函数，支持返回值中断传播。系统定义的主要事件：gamebegin（游戏开始）、level_play（进入关卡）、level_next（下一关）、level_back（返回）、show_setup（设置）、onLevelChange（关卡进度变化）、onpowerChange（体力变化）、onChapterChange（章节变化）。

3.3 数据管理器 datamgr.cs

datamgr 采用单例模式，基于 Luban 配置表框架加载 JSON 配置数据。支持编辑器中拖拽 TextAsset 和运行时读取文件两种加载方式。提供 GetChapter(id) 和 GetLevel(id) 接口。

```
public class datamgr : MonoBehaviour {

    public TextAsset tbchapter, tblevel;

    static datamgr _instance;

    public static datamgr Instance {

        get { if (_instance == null) {

            var x = Instantiate(Resources.Load("DataManager")) as GameObject;

            _instance = x.GetComponent<datamgr>(); } return _instance; } }

    public Tables Tables { get; private set; }

    public void LoadConfigTables() {

        Func<string, JSONNode> loader = file => {

            TextAsset ta = file=="tbchapter"?tbchapter:file=="tblevel"?tblevel:null;

            if (ta != null) return JSON.Parse(ta.text);

            string fp = Path.Combine(Application.dataPath, "data", file+".json");

            return File.Exists(fp) ? JSON.Parse(File.ReadAllText(fp)) : null; };

        Tables = new Tables(loader);

    }

    public DrChapter GetChapter(int id) => Tables?.TbChapter?.GetOrDefault(id);

    public DrLevel GetLevel(int id) => Tables?.TbLevel?.GetOrDefault(id);

}
```

3.4 玩家数据 PlayerData.cs

管理玩家游戏进度和资源数据，包括当前关卡、已解锁关卡、章节、体力值。数据以 JSON 持久化到 persistentDataPath/playerData.json。体力系统每 15 分钟自动恢复 1 点，上限 100，支持离线累积。

```
public class GameData {

    JsonData data;

    public GameData(JsonData d) { data = d; }

    internal int levelid {

        get => int.Parse(data.Has("levelid"?data["levelid"].ToString():"100001"));

        set { data["levelid"]=value.ToString(); Main.DispEvent("onLevelChange"); } }

    internal int currChapter {

        get => int.Parse(data.Has("currChapter"?data["currChapter"].ToString():"1");

        set { data["currChapter"]=value.ToString(); Main.DispEvent("onChapterChange"); } }

    internal int power {

        get {

            int cur = int.Parse(data.Has("power"?data["power"].ToString():"100");

            long now = DateTime.Now.Ticks / TimeSpan.TicksPerSecond;
```



```

        long elapsed = now - lastpowerUpdateTime;

        int recv = (int)(elapsed / 900);

        if (recv > 0) { cur = Math.Min(100, cur+recv);

            data["power"] = cur.ToString(); lastpowerUpdateTime = now; }

        return cur; }

set { data["power"] = Math.Min(100, value).ToString();

    lastpowerUpdateTime = DateTime.Now.Ticks/TimeSpan.TicksPerSecond;

    Main.DispEvent("onpowerChange"); } }

internal long lastpowerUpdateTime {

    get => long.Parse(data.Has("lastpowerUpdateTime")

        ?data["lastpowerUpdateTime"].ToString()

        :(DateTime.Now.Ticks/TimeSpan.TicksPerSecond).ToString());

    set => data["lastpowerUpdateTime"] = value.ToString(); }

public bool isOpened(int id) {

    var arr = data["opened"];

    for(int i=0;i<arr.Count;i++) if(int.Parse(arr[i].ToString())==id) return true;

    return false; }

public void Open(int id) { data["opened"].Add(id.ToString());

    Main.DispEvent("onLevelChange"); }

}

public class PlayerData : MonoBehaviour {

    public static GameData gd;

    void Awake() { loadData();

        Main.RegistEvent("onLevelChange", x => { saveData(); return 1; });

        Main.RegistEvent("onpowerChange", x => { saveData(); return null; }); }

    void loadData() {

        var pa = Application.persistentDataPath+"/playerData.json";

        if(File.Exists(pa)) try {

            gd = new GameData(JsonMapper.ToObject<JsonData>(File.ReadAllText(pa)));

        } catch { gd = new GameData(new JsonData()); }

        else gd = new GameData(new JsonData()); }

    public void saveData() {

        File.WriteAllText(Application.persistentDataPath+"/playerData.json",

            JsonMapper.ToJson(gd.getData())); }

}

```

3.5 拼图管理器 picmgr.cs

picmgr 是拼图核心管理类，负责图片切割、碎片创建、打乱算法、边框刷新、胜利检测。根据 DrLevel 配置的 LevelFigureX（列数）和 LevelFigureY（行数），将原图均匀切割为碎片。每个碎片使用 RawImage 通过 UV 矩形控制显示区域。

```

public class picmgr : MonoBehaviour {

```



```

public Texture2D pic; public int width=3, height=3;

internal static picmgr instance;

internal float carWid => trans.rect.width/width;

internal float carHei => trans.rect.height/height;

RectTransform trans => GetComponent<RectTransform>();

public IEnumerator LoadLevel(DrLevel lv) {

    width=lv.LevelFigureX; height=lv.LevelFigureY;

    pic = Resources.Load(lv.LevelFigure) as Texture2D;

    yield return StartCoroutine(

        CreateGridImages(lv.Id, lv.OutOfPlaceNumber, lv.DifficultyTier==2));

}

public IEnumerator CreateGridImages(int lv=1, int maxKeep=-1, bool hard=false) {

    clearOld();

    Vector2 sz = trans.rect.size;

    float cw=sz.x/width, ch=sz.y/height;

    float delay = 1f/(width*height);

    for(int x=0;x<width;x++) for(int y=0;y<height;y++) {

        yield return new WaitForSeconds(delay);

        create(x, y, cw, ch, hard, false); }

    yield return new WaitForSeconds(0.3f);

    ShuffleGridPositions(lv, maxKeep, hard);

    for(int i=0; i<transform.childCount; i++)

        transform.GetChild(i).GetComponent<DraggableGridItem>().Turn();

}

void create(int x, int y, float cw, float ch, bool hard, bool bed) {

    var go = Instantiate(Resources.Load("GridCell")) as GameObject;

    go.name = $"GridCell_{x}_{y}"; go.transform.SetParent(transform, false);

    var rt = go.GetComponent<RectTransform>();

    var dg = go.GetComponent<DraggableGridItem>();

    dg.pic = pic; dg.canvas = GetComponentInParent<Canvas>();

    dg.uvX = (float)x/width; dg.uvY = (float)y/height;

    dg.uvWidth = 1f/width; dg.uvHeight = 1f/height;

    dg.PositionIndex = x*height + y;

    rt.anchorMin=rt.anchorMax=rt.pivot=Vector2.zero;

    rt.sizeDelta = new Vector2(cw, ch);

    if(bed) { rt.anchoredPosition=new Vector2(x*cw, y*ch); dg.Turn(true); }

    else rt.anchoredPosition=Vector2.zero;

    CreateBorders(go, cw, ch);

    rt.DOAnchorPos(new Vector2(x*cw, y*ch), 0.2f);

}

```

打乱算法使用确定性随机种子（seed = levelId*1000 + (isHard?1:0)），普通模式保留约 30% 原位，困难模式通过曼哈顿距离贪心优化使碎片尽量远离原位。


```

public void ShuffleGridPositions(int lv, int maxKeep, bool hard) {

    var children = new List<RectTransform>();

    var origPos = new List<Vector2>();

    var origGrid = new List<Vector2Int>();

    foreach(Transform c in transform) {

        var r=c.GetComponent<RectTransform>(); if(r==null) continue;

        children.Add(r); origPos.Add(r.anchoredPosition);

        var p=c.name.Replace("GridCell_", "").Split('_');

        if(p.Length==2&&int.TryParse(p[0], out int x)&&int.TryParse(p[1], out int y))

            origGrid.Add(new Vector2Int(x, y));

        else origGrid.Add(new Vector2Int(-1, -1)); }

    if(children.Count<2) return;

    var avail=new List<int>(); for(int i=0; i<children.Count; i++) avail.Add(i);

    int seed=lv*1000+(hard?1:0);

    var rng=new System.Random(seed);

    if(maxKeep==1) maxKeep=Mathf.Max(0, Mathf.RoundToInt(children.Count*0.3f));

    maxKeep=Mathf.Clamp(maxKeep, 0, children.Count-1);

    int keepCt=rng.Next(0, maxKeep+1);

    var ai=new List<int>(); for(int i=0; i<children.Count; i++) ai.Add(i);

    var sr=new System.Random(seed+1);

    for(int i=ai.Count-1; i>0; i--) { int j=sr.Next(0, i+1);

        (ai[i], ai[j])=(ai[j], ai[i]); }

    var keepIdx=ai.GetRange(0, keepCt);

    foreach(int k in keepIdx) avail.Remove(k);

    var target=new List<int>(avail);

    if(hard&&origGrid.Count==children.Count) {

        var dr=new System.Random(seed+3);

        for(int a=0; a<3; a++) for(int i=0; i<target.Count-1; i++) {

            int ci=avail[i], ct=target[i];

            int cd=Mathf.Abs(origGrid[ci].x-origGrid[ct].x)+

                Mathf.Abs(origGrid[ci].y-origGrid[ct].y);

            for(int j=i+1; j<target.Count; j++) {

                int oi=avail[j], ot=target[j];

                int nd1=Mathf.Abs(origGrid[ci].x-origGrid[ot].x)+

                    Mathf.Abs(origGrid[ci].y-origGrid[ot].y);

                int nd2=Mathf.Abs(origGrid[oi].x-origGrid[ct].x)+

                    Mathf.Abs(origGrid[oi].y-origGrid[ct].y);

                int od=Mathf.Abs(origGrid[oi].x-origGrid[ot].x)+

                    Mathf.Abs(origGrid[oi].y-origGrid[ot].y);

                if(nd1>cd&&nd2>od) {(target[i], target[j])=(target[j], target[i]); break;} } }

        else { var tr=new System.Random(seed+2);

            for(int i=target.Count-1; i>0; i--) { int j=tr.Next(0, i+1);

                (target[i], target[j])=(target[j], target[i]); } }

```



```

for (int i=0; i<children.Count; i++) {

    if (keepIdx.Contains(i)) continue;

    int idx=avail.IndexOf(i);

    if (idx>0&&idx<target.Count) children[i].anchoredPosition=origPos[target[idx]]; }

RefreshAllPositionIndices(); UpdateBorderVisibility();

}

```

边框刷新算法（UpdateBorderVisibility）分为两组独立处理：非拖拽组（无参重载）和拖拽组（List 参数重载）。每组只在其组内搜索邻居，确保拖拽中碎片的边框正确反映组内连接关系。

```

public void UpdateBorderVisibility() {

    var all=new List<DraggableGridItem>();

    foreach(Transform c in transform) {

        var it=c.GetComponent<DraggableGridItem>();

        if(it!=null&&!it.IsBeingDragged) all.Add(it); }

    if(all.Count>0) UpdateBorderVisibility(all); }

public void UpdateBorderVisibility(List<DraggableGridItem> group) {

    if(group==null||group.Count==0) return;

    Vector2 sz=trans.rect.size;

    float cw=sz.x/width, ch=sz.y/height, th=1.5f;

    foreach(var item in group) {

        var rt=item.GetComponent<RectTransform>(); if(rt==null) continue;

        var p=item.name.Replace("GridCell_", "").Split('_');

        if(p.Length!=2) continue; int x=int.Parse(p[0]), y=int.Parse(p[1]);

        var rN=F(rt.anchoredPosition+new Vector2(cw, 0), group, th);

        var lN=F(rt.anchoredPosition-new Vector2(cw, 0), group, th);

        var tN=F(rt.anchoredPosition+new Vector2(0, ch), group, th);

        var bN=F(rt.anchoredPosition-new Vector2(0, ch), group, th);

        var trN=F(rt.anchoredPosition+new Vector2(cw, ch), group, th);

        var tlN=F(rt.anchoredPosition+new Vector2(-cw, ch), group, th);

        var brN=F(rt.anchoredPosition+new Vector2(cw, -ch), group, th);

        var blN=F(rt.anchoredPosition+new Vector2(-cw, -ch), group, th);

        bool rc=rN!=null&&rN.name=="GridCell_{x+1}_{y}";

        bool lc=lN!=null&&lN.name=="GridCell_{x-1}_{y}";

        bool tc=tN!=null&&tN.name=="GridCell_{x}_{y+1}";

        bool bc=bN!=null&&bN.name=="GridCell_{x}_{y-1}";

        bool trc=trN!=null&&trN.name=="GridCell_{x+1}_{y+1}";

        bool tlc=tlN!=null&&tlN.name=="GridCell_{x-1}_{y+1}";

        bool brc=brN!=null&&brN.name=="GridCell_{x+1}_{y-1}";

        bool blc=blN!=null&&blN.name=="GridCell_{x-1}_{y-1}";

        item.adjacentRight=rc; item.adjacentLeft=lc;

        item.adjacentTop=tc; item.adjacentBottom=bc;

        SB(item, "RightBorder", !rc); SB(item, "LeftBorder", !lc);
    }
}

```



```

        SB(item, "TopBorder", !tc); SB(item, "BottomBorder", !bc);

        SB(item, "TopLeftBorder", !tc||!lc||!tlc);

        SB(item, "TopRightBorder", !tc||!rc||!trc);

        SB(item, "BottomRightBorder", !bc||!rc||!brc);

        SB(item, "BottomLeftBorder", !bc||!lc||!blc);

    }

}

DraggableGridItem F(Vector2 p, List<DraggableGridItem> items, float t) {

    foreach(var i in items) { var r=i.GetComponent<RectTransform>();

        if(r!=null&&Vector2.Distance(r.anchoredPosition,p)<=t) return i; } return null; }

void SB(GameObject p, string n, bool v) {

    var c=p.transform.Find(n); if(c!=null) c.gameObject.SetActive(v); }

```

3.6 拖拽系统 DraggableGridItem.cs

实现拼图碎片的完整拖拽交互（IBeginDragHandler、IDragHandler、IEndDragHandler）。支持连通组整体拖拽（DFS 收集正确连接的碎片）、边界限制防止出界、DOTween 动画交换位置。

```

public class DraggableGridItem : MonoBehaviour,

    IBeginDragHandler, IDragHandler, IEndDragHandler {

    public Canvas canvas;

    static int origIdx; public static int targetIdx;

    public int PositionIndex { get; set; }

    public bool IsBeingDragged => isDrag; bool isDrag;

    public static bool isAnyItemDrag = false;

    public bool adjL, adjR, adjT, adjB;

    List<DraggableGridItem> grp;

    Dictionary<DraggableGridItem, Vector2> gp;

    internal Texture2D pic; internal float uvX, uvY, uvW, uvH;

    public RectTransform rt => GetComponent<RectTransform>();

    public void OnBeginDrag(PointerEventData e) {

        if(isAnyItemDrag) return; isAnyItemDrag=true;

        origIdx=PositionIndex; Collect(); isDrag=true;

        if(grp!=null) foreach(var it in grp) it.transform.SetAsLastSibling(); }

    public void OnDrag(PointerEventData e) {

        var pr=transform.parent as RectTransform; if(pr==null) return;

        var all=grp??new List<DraggableGridItem>() {this};

        var rd=rt.anchoredPosition+e.delta/canvas.scaleFactor-OP();

        float mnX=float.MinValue, mxX=float.MaxValue, mnY=float.MinValue, mxY=float.MaxValue;

        foreach(var it in all) {

            var o=it==this?OP():(gp!=null&&gp.TryGetValue(it, out var ov)?ov:it.rt.anchoredPosition);

            float iw=it.rt.rect.width, ih=it.rt.rect.height;

            mnX=Mathf.Max(mnX, -o.x);mxX=Mathf.Min(mxX, pr.rect.width-iw-o.x);

```



```

        mnY=Mathf.Max(mnY,-o.y);mxY=Mathf.Min(mxY,pr.rect.height-ih-o.y); }

var d=new Vector2(Mathf.Clamp(rd.x,mnX,mxX),Mathf.Clamp(rd.y,mnY,mxY));

rt.anchoredPosition=OP()+d;

if(grp!=null&&gp!=null) foreach(var it in grp)

    if(it!=this&&gp.TryGetValue(it,out var ov)) it.rt.anchoredPosition=ov+d;

if(picmgr.instance!=null) { picmgr.instance.UpdateBorderVisibility();

    if(grp!=null) picmgr.instance.UpdateBorderVisibility(grp); }

var df=rt.anchoredPosition-CP();

if(Mathf.Abs(df.x)>cw()/2||Mathf.Abs(df.y)>ch()/2) {

    int x=Mathf.RoundToInt(rt.anchoredPosition.x/cw());

    int y=Mathf.RoundToInt(rt.anchoredPosition.y/ch());

    targetIdx=Mathf.Clamp(x,0,picmgr.instance.width)*picmgr.instance.height

        +Mathf.Clamp(y,0,picmgr.instance.height); }

else targetIdx=PositionIndex; }

public void OnEndDrag(PointerEventData e) {

    if(origIdx!=targetIdx) Swap(TP());

    else Reset();

    grp=null;gp=null;isDrag=false;isAnyItemDrag=false; }

Vector2 OP() => new((origIdx/picmgr.instance.width)*cw(),

    (origIdx%picmgr.instance.height)*ch());

Vector2 TP() => new((targetIdx/picmgr.instance.width)*cw(),

    (targetIdx%picmgr.instance.height)*ch());

Vector2 CP() => new((PositionIndex/picmgr.instance.width)*cw(),

    (PositionIndex%picmgr.instance.height)*ch());

float cw()=>picmgr.instance.carWid; float ch()=>picmgr.instance.carHei;

void Collect() {

    grp=new();gp=new();

    var map=new Dictionary<(int,int),DraggableGridItem>();

    foreach(Transform c in transform.parent) {

        var it=c.GetComponent<DraggableGridItem>();

        if(it!=null&&TP(c.name,out int cx,out int cy)) map[(cx,cy)]=it; }

    if(!TP(name,out int sx,out int sy)) { grp.Add(this); return; }

    void dfs(int x,int y) {

        if(!map.TryGetValue((x,y),out var cur)||grp.Contains(cur)) return;

        grp.Add(cur); gp[cur]=cur.rt.anchoredPosition;

        if(cur.adjL) dfs(x-1,y); if(cur.adjR) dfs(x+1,y);

        if(cur.adjT) dfs(x,y+1); if(cur.adjB) dfs(x,y-1); }

    dfs(sx,sy); }

bool TP(string n,out int x,out int y) { x=y=-1;

    if(!n.StartsWith("GridCell_")) return false;

    var p=n.Replace("GridCell_", "").Split('_');

    return p.Length==2&&int.TryParse(p[0],out x)&&int.TryParse(p[1],out y); }

```



```

void Swap(Vector2 tp) { /* animation */ }

void Reset() { /* reset */ }

public void Turn(bool at=false) { /* flip */ }

}

```

3.7 卡片系统 card.cs

card 用于章节选择界面的关卡卡片，根据关卡状态显示不同内容。已通关卡片执行翻转动画，未解锁显示背面和编号。

```

public class card : MonoBehaviour {

    public Texture2D back; internal int levelid;

    internal float uvX, uvY, uvW, uvH; internal Texture texture;

    public TextMeshProUGUI level; public bool isTurning=false;

    RawImage ri => GetComponent<RawImage>();

    internal void Load() {

        if(levelid<PlayerData.gd.levelid) {

            if(PlayerData.gd.isOpened(levelid)) {

                ri.uvRect=new Rect(uvX, 1-uvY-uvH, uvW, uvH); ri.texture=texture;

                level.gameObject.SetActive(false); }

            else { DOVirtual.DelayedCall(1f, ()=>{

                ri.uvRect=new Rect(0, 0, 1, 1); ri.texture=back; isTurning=true;

                transform.DOScaleX(0.5f).OnComplete(()=>{

                    ri.uvRect=new Rect(uvX, 1-uvY-uvH, uvW, uvH); ri.texture=texture;

                    transform.DOScaleX(1.5f).OnComplete(()=>{isTurning=false;});});

                PlayerData.gd.Open(levelid); }); } }

            else { ri.uvRect=new Rect(0, 0, 1, 1); ri.texture=back;

                level.gameObject.SetActive(true); } } }

```

3.8 边框系统 BorderSpriteLoader.cs

BorderSpriteLoader 负责为九宫格边框子对象加载贴图，支持运行时动态切换。贴图从 Resources/Card 加载，具有静态缓存避免重复加载。RefreshBasedOnFlags 根据方向连接状态切换角贴图。

四、用户界面设计

4.1 章节选择界面

章节选择界面以网格形式展示当前章节的所有关卡卡片，每张卡片使用 `card` 组件显示章节预览图的对应区域。顶部显示关卡编号和体力值。点击「开始」按钮触发 `level_play` 事件进入游戏。界面通过事件 `gamebegin`、`onLevelChange`、`onChapterChange`、`level_next` 驱动刷新。

4.2 游戏界面

游戏界面承载拼图玩法。加载关卡后调用 `picmgr.LoadLevel` 进行图片切割打乱，碎片以动画方式显示。显示关卡编号和难度标签，提供返回章节、设置、下一关（通关后出现）按钮。

4.3 设置界面

设置界面提供游戏初始化（删除存档重启）和体力购买（测试用途）。所有 UI 窗体继承自 `frmbase` 基类，实现统一 `show/hide` 生命周期管理。

五、数据配置设计

5.1 章节配置

DrChapter: Id (章节 ID)、ChapterTitle (标题)、ChapterFigure (预览图资源名)、ChapterFigureX/ChapterFigureY (行列数)、NextChapter (下一章)、LevelId (关卡 ID 列表)。

5.2 关卡配置

DrLevel: Id (关卡 ID)、NextLevel (下一关)、LevelFigure (图片资源)、LevelFigureX/LevelFigureY (切割行列)、DifficultyTier (难度 1普通/2困难)、OutOfPlaceNumber (最多原位保留数)、MDistanceRange (曼哈顿距离范围)。配置经 Luban 从 Excel 导出为 JSON。

六、核心算法说明

6.1 拼图打乱算法

确定性种子 ($\text{levelId} * 1000 + \text{isHardFlag}$) 保证同一关卡结果一致。Fisher-Yates 洗牌随机排列碎片，普通模式保留 30% 原位，困难模式通过贪心迭代最大化碎片偏离原位的曼哈顿距离（3 轮优化）。

6.2 连通组检测算法

DFS 深度优先搜索：从当前碎片出发，沿 `adjacent` 标志为 `true` 的方向递归扩展。坐标映射通过碎片名称 `GridCell_x_y` 解析，构建坐标到碎片的字典。一组内所有碎片保持正确相对位置。

6.3 位置交换算法

计算拖拽组目标位置 → 识别被覆盖碎片和空出位置 → 按网格顺序排序后建立映射 → `DOTween Sequence` 并行动画 (0.25s) → 完成回调刷新索引、边框和胜利检测。

6.4 胜利检测算法

遍历碎片，基于实际 `anchoredPosition` 检测四个方向是否存在名称匹配的邻居。所有方向均匹配则判定胜利，触发界面向上位移动画并显示下一关按钮。

6.5 边框刷新算法

分两组独立刷新：非拖拽组和拖拽组。每组只在组内搜索邻居（位置阈值 1.5px）。正确连接隐藏边框，否则显示。角贴图根据四方向连接状态自动切换（角、边、反转角三种贴图）。

七、运行环境

硬件：ARM64/x86_64 1.5GHz+、2GB+ 内存、100MB 存储、1280x720+ 分辨率。

软件：iOS 13.0+ / Android 8.0+、Unity 2022.3 LTS。

依赖：DOTween、LitJson、Luban。